

Unit 1

What is OOP in Java?

OOP stands for **Object-Oriented Programming**. It is a programming paradigm based on the concept of "**objects**", which can contain data (fields or attributes) and code (methods or functions).

Java is a **pure object-oriented programming language**, which means it uses OOP principles to design programs.

Core Concepts of OOP in Java

1. **Class**
A blueprint for creating objects. It defines properties and behaviors.
Example: `class Car { ... }`
 2. **Object**
An instance of a class. It represents a real-world entity with state and behavior.
 3. **Encapsulation**
Wrapping data (variables) and code (methods) together and restricting direct access using access modifiers (`private`, `public`, etc.).
 4. **Inheritance**
A mechanism where one class inherits properties and methods from another class, promoting code reuse.
 5. **Polymorphism**
The ability to take many forms. It allows one interface to be used for a general class of actions, and the exact action is determined by the specific instance.
 6. **Abstraction**
Hiding complex implementation details and showing only the necessary features of an object.
-

Why OOP?

- **Modularity** — Code is organized into classes and objects.
- **Reusability** — Inheritance and polymorphism promote code reuse.
- **Maintainability** — Encapsulation makes it easier to manage code.
- **Scalability** — Easier to add new features without breaking existing code.

Simple Example

Java

```
class Dog {
    String name;

    void bark() {
        System.out.println(name + " says Woof!");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog1 = new Dog();
        dog1.name = "Buddy";
        dog1.bark(); // Output: Buddy says Woof!
    }
}
```

Q.Explain the properties of oop

1. Encapsulation

- **Definition:** Wrapping data (variables) and methods (functions) into a single unit called a class.
- **Purpose:** Protects the data from outside interference and misuse by restricting access using **access modifiers** (private, public, protected).
- **Example:** Using private fields with public getter/setter methods.
- **Benefit:** Improves security and hides internal object details.

2. Inheritance

- **Definition:** A mechanism where a new class (child/subclass) inherits properties and behaviors (fields and methods) from an existing class (parent/superclass).
- **Purpose:** Promotes **code reuse** and establishes a hierarchical relationship.
- **Example:**

```
java

class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}
```

```

    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}

```

- **Benefit:** Avoids rewriting code, easier to maintain and extend.

3. Polymorphism

- **Definition:** The ability of a variable, function, or object to take multiple forms.
- **Types:**
 - **Compile-time (Method Overloading):** Same method name, different parameters.
 - **Run-time (Method Overriding):** Subclass provides its own implementation of a method from the superclass.
- **Example:**

```

java

class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}

```

- **Benefit:** Makes code flexible and extensible.

4. Abstraction

- **Definition:** Hiding complex implementation details and showing only essential features to the user.
- **Purpose:** Simplifies usage and interface by focusing on "what" an object does instead of "how" it does it.
- **Implemented Using:**
 - **Abstract classes**
 - **Interfaces**
- **Example:**

```

java

abstract class Shape {
    abstract void draw();
}
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing Circle");
    }
}

```

- **Benefit:** Reduces complexity, improves security by hiding internal logic.

Summary Table:

Property	Description	Benefit
Encapsulation	Wrapping data & methods, hiding details	Data protection & modularity
Inheritance	Child class inherits from parent class	Code reuse & hierarchy
Polymorphism	Same entity takes multiple forms	Flexibility & extensibility
Abstraction	Hide complexity, show essential features	Simplifies interface & security

Q.What is IVM (Instance Variable Memory)?

- **Instance Variable Memory** refers to the memory allocated to the **instance variables** (non-static variables) of a class **when an object is created**.
 - Each object has its **own copy** of instance variables.
 - The memory for instance variables is allocated in the **heap memory**.
 - Instance variables are used to maintain the **state** of each object separately.
-

Explanation with Diagram

Consider a class Car:

```

java
Copy code
class Car {
    int speed;        // instance variable
    String color;     // instance variable

    void display() {
        System.out.println("Speed: " + speed + ", Color: " + color);
    }
}

```

```
}
```

When you create two objects:

```
java
Copy code
Car car1 = new Car();
Car car2 = new Car();

car1.speed = 50;
car1.color = "Red";

car2.speed = 70;
car2.color = "Blue";
```

Memory Representation Diagram

lua

Heap Memory:

```
+-----+
| Object car1 |
| ----- |
| speed = 50  |
| color = "Red" |
+-----+

+-----+
| Object car2 |
| ----- |
| speed = 70  |
| color = "Blue" |
+-----+
```

Stack Memory:

```
car1 --> Reference to Object car1 in heap
car2 --> Reference to Object car2 in heap
```

Summary

- **Instance variables** belong to objects, not the class.
- Each object gets its **own memory space** for instance variables in the **heap**.
- **References** to objects are stored in the **stack**.

What is Bytecode?

- **Bytecode** is an intermediate, platform-independent code generated by the **Java compiler** after compiling a `.java` source file.
 - Instead of converting directly into machine-specific code, Java source code is compiled into **bytecode**, which is stored in `.class` files.
 - This bytecode is **not human-readable** but is designed to be executed by the **Java Virtual Machine (JVM)**.
 - JVM interprets (or just-in-time compiles) the bytecode into machine code specific to the host operating system and hardware.
-

Why Bytecode?

- **Portability:** Write once, run anywhere (WORA). Bytecode can run on any machine that has a compatible JVM.
 - **Security:** Bytecode runs inside the JVM sandbox, providing a secure execution environment.
 - **Optimization:** JVM can optimize bytecode execution at runtime.
-

Summary

Term	Description
Source Code	Java <code>.java</code> files written by the programmer
Bytecode	Platform-independent <code>.class</code> files generated by the compiler
JVM	Java Virtual Machine that executes bytecode on any platform

Quick Example:

bash

```
javac HelloWorld.java    # Compiles Java source to HelloWorld.class (bytecode)
java HelloWorld          # JVM runs the bytecode on your machine
```

What is a Flowchart?

- A **flowchart** is a **visual diagram** that represents the step-by-step flow or process of an algorithm or program.
- It uses different **symbols** to show different types of actions or steps.
- Helps in planning, designing, and understanding how a program or process works.

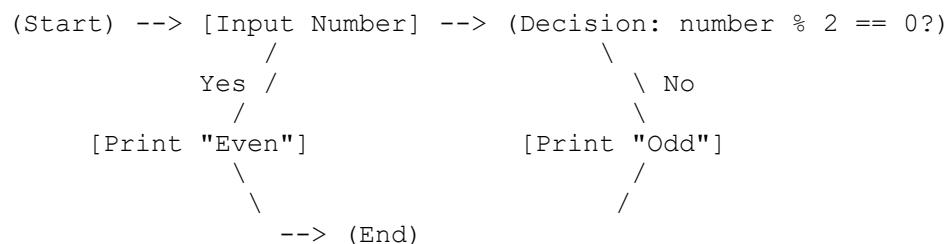
Common Flowchart Symbols:

Symbol	Meaning
Oval	Start or End
Rectangle	Process or Instruction
Parallelogram	Input or Output
Diamond	Decision (Yes/No or True/False)
Arrow	Flow of control/Direction

Simple Flowchart Example:

Flowchart to check if a number is even or odd

mathematica



Different Data Types in Java

In programming, **data types** specify the type of data a variable can hold. Java has two main categories:

1. Primitive Data Types

Stores simple values.

Data Type	Size	Description	Example
byte	1 byte	Small integer (-128 to 127)	byte b = 100;
short	2 bytes	Small integer (-32,768 to 32,767)	short s = 1000;
int	4 bytes	Integer	int i = 10000;
long	8 bytes	Large integer	long l = 100000L;
float	4 bytes	Floating-point number	float f = 3.14f;
double	8 bytes	Double-precision floating-point	double d = 3.14159;

Data Type	Size	Description	Example
char	2 bytes	Single Unicode character	char c = 'A';
boolean	1 bit	True or false	boolean flag = true;

2. Non-Primitive Data Types (Reference Types)

Objects and arrays, e.g., `String`, arrays, classes.

Example:

```
java

String name = "King";
int[] numbers = {1, 2, 3};
```

Unit 2

What is a Data Type?

- A **data type** defines the **type of data** a variable can hold.
 - It tells the compiler how much memory to allocate and what kind of operations can be performed on that data.
 - Every variable in Java must have a declared data type.
-

Different Data Types in Java

Java has two main categories of data types:

1. Primitive Data Types

These store **simple values** and are predefined by the language.

Data Type	Size	Description	Example
byte	1 byte	Small integer (-128 to 127)	byte b = 100;
short	2 bytes	Small integer (-32,768 to 32,767)	short s = 1000;
int	4 bytes	Integer numbers	int i = 10000;
long	8 bytes	Large integers	long l = 100000L;
float	4 bytes	Floating-point number	float f = 3.14f;

Data Type	Size	Description	Example
double	8 bytes	Double-precision floating-point	<code>double d = 3.14159;</code>
char	2 bytes	Single Unicode character	<code>char c = 'A';</code>
boolean	1 bit	True or false	<code>boolean flag = true;</code>

2. Non-Primitive Data Types (Reference Types)

- These store **objects** and **arrays**.
- Examples include `String`, arrays, classes, and interfaces.
- They refer to memory locations where data is stored, not the actual data itself.

Example:

```
java
Copy code
String name = "King";
int[] numbers = {1, 2, 3};
```

Summary:

Category	Description	Examples
Primitive Types	Store simple values	int, float, char, boolean
Reference Types	Store references to objects	String, Array, Class objects

s

Unit 4

Q.1what is inheritance .explain diff type of inheritance with block diagram

ans>

Inheritance in Java is a mechanism where one class (called **subclass** or **child class**) acquires the properties and behaviors (fields and methods) of another class (called **superclass** or **parent class**). It promotes **code reusability** and supports **method overriding** for runtime polymorphism.

Syntax of Inheritance in Java:

```
class SuperClass {  
    // fields and methods  
}
```

```
class SubClass extends SuperClass {  
    // additional fields and methods  
}
```

Types of Inheritance in Java:

Java supports the following types of inheritance:

1. Single Inheritance

- One subclass inherits from one superclass.

Diagram:

```
Animal (Superclass)  
  ↑  
Dog (Subclass)
```

Example:

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking...");  
    }  
}
```

2. Multilevel Inheritance

- A class is derived from a derived class.

Diagram:

Animal
↑
Dog
↑
Puppy

Example:

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Barking...");  
    }  
}
```

```
class Puppy extends Dog {  
    void weep() {  
        System.out.println("Weeping...");  
    }  
}
```

}3. Hierarchical Inheritance

- Multiple classes inherit from a single superclass.

Diagram:

markdown
Copy code
Animal
↑ ↑
Dog Cat

Example:

```
class Animal {  
    void eat() {  
        System.out.println("Eating...");  
    }  
}
```

```

class Dog extends Animal {
    void bark() {
        System.out.println("Barking...");
    }
}

class Cat extends Animal {
    void meow() {
        System.out.println("Meowing...");
    }
}

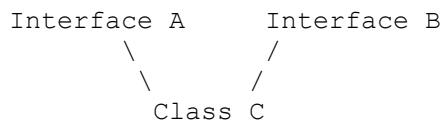
```

4. Multiple Inheritance (via Interface only in Java)

- Java doesn't support multiple inheritance with classes to avoid ambiguity (Diamond problem). But **it supports multiple inheritance through interfaces**.

Diagram:

vbnet



Example:

```

java
Copy code
interface A {
    void show();
}

interface B {
    void display();
}

class C implements A, B {
    public void show() {
        System.out.println("Show method");
    }

    public void display() {
        System.out.println("Display method");
    }
}

```

Q.2 , Explain keyword (a.super, b. final, c. extends)?

super keyword

► Used to refer to:

1. **Parent class constructor**
2. **Parent class method**
3. **Parent class variable**

Example 1: Access parent class method

```
java
Copy code
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    void sound() {
        super.sound(); // calls parent class method
        System.out.println("Dog barks");
    }
}
```

Example 2: Access parent class constructor

```
java
Copy code
class Animal {
    Animal() {
        System.out.println("Animal constructor called");
    }
}

class Dog extends Animal {
    Dog() {
        super(); // calls Animal constructor
        System.out.println("Dog constructor called");
    }
}
```

final keyword

► Used for:

1. **Variables** – Value cannot be changed (constant)
 2. **Methods** – Cannot be overridden
 3. **Classes** – Cannot be inherited
-

✅ Example 1: Final variable

```
java
Copy code
final int x = 10;
// x = 20; // ❌ Error: cannot change final variable
```

✅ Example 2: Final method

```
java
Copy code
class Animal {
    final void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal {
    // void eat() {} ❌ Error: Cannot override final method
}
```

✅ Example 3: Final class

```
java
Copy code
final class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

// class Dog extends Animal {} ❌ Error: Cannot inherit final class
```

📄 **extends** keyword

➤ Used to inherit a class

It means "this class is a child of that class".

Example:

```
java
Copy code
class Animal {
    void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal { // Dog inherits Animal
    void bark() {
        System.out.println("Barking...");
    }
}
```

What is a Package in Java?

A **package** is a group of related classes and interfaces.
It helps:

- Avoid **name conflicts**
- Organize classes **neatly**
- **Control access** with protected/private modifiers

Types of Packages

1. **Built-in packages** (like `java.util`, `java.io`, etc.)
2. **User-defined packages** (created by you)

Program: Creating Your Own Package

File Structure:

```
mathematica
Copy code
MyPackageDemo/
├── mypack/
│   └── Message.java
└── TestPackage.java
```

◆ Step 1: Create a class inside a package

mypack/Message.java

```
java
Copy code
package mypack;

public class Message {
    public void show() {
        System.out.println("Hello King! This is your own package.");
    }
}
```

◆ Step 2: Access the package in another class

TestPackage.java

```
java
Copy code
import mypack.Message;

class TestPackage {
    public static void main(String args[]) {
        Message m = new Message();
        m.show();
    }
}
```

How to Compile and Run

 Open terminal or CMD:

```
bash
cd MyPackageDemo
javac mypack/Message.java      # Compile package class
javac TestPackage.java        # Compile main class
java TestPackage               # Run main class
```

If you follow this, the output will be:

```
kotlin
Copy code
Hello King! This is your own package.
```


What is Abstract in Java?

- `abstract` is a **keyword** used with **classes** and **methods**.
 - It is used when **we want to define a general concept**, but let **child classes provide the actual implementation**.
-

Abstract Class:

- A class declared with the `abstract` keyword.
 - It **cannot be instantiated** (you can't create an object of it).
 - It can **contain both abstract and non-abstract methods**.
-

Abstract Method:

- A method declared without a body.
 - It must be **overridden in the child class**.
-

Syntax:

```
java
Copy code
abstract class Animal {
    abstract void sound(); // abstract method
    void sleep() {          // regular method
        System.out.println("Sleeping...");
    }
}
```

Example: Abstract Class in Action

```
java
Copy code
// Abstract class
abstract class Animal {
    abstract void sound(); // abstract method

    void sleep() {
```

```

        System.out.println("Sleeping...");
    }
}

// Subclass
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

// Main class
class TestAbstract {
    public static void main(String[] args) {
        Animal a = new Dog(); // polymorphism
        a.sound();             // Output: Dog barks
        a.sleep();             // Output: Sleeping...
    }
}

```

Rules:

- If a class has at least one abstract method, the class **must be abstract**.
 - You **can't create an object** of an abstract class.
 - Subclasses **must override** all abstract methods.
-

Difference Between `Abstract Class` and `Interface` in Java

Feature	Abstract Class	Interface
Keyword	<code>abstract</code>	<code>interface</code>
Methods	Can have abstract and non-abstract	Can have only abstract methods (till Java 7) Java 8+ supports default & static methods
Variables	Can have variables (non-final)	Only <code>public static final</code> (constant) variables
Constructors	Can have constructors	 Cannot have constructors
Multiple Inheritance	 Not supported	 Supported (a class can implement multiple interfaces)
Access Modifiers	Can be <code>public</code> , <code>protected</code> , etc.	All methods are <code>public</code> by default

Feature	Abstract Class	Interface
Use Case	For shared base code with some abstraction	For defining pure abstraction/blueprints
Instantiation	❌ Cannot instantiate	❌ Cannot instantiate

✅ Abstract Class Example:

```
java
Copy code
abstract class Animal {
    abstract void sound();
    void eat() {
        System.out.println("Eating...");
    }
}
```

✅ Interface Example:

```
java
Copy code
interface Animal {
    void sound(); // implicitly public and abstract
}
```

🔥 Summary (Easy Tips):

Think Abstract Class	Think Interface
"What it is "	"What it does "
Partial abstraction	Full abstraction
Is-A relationship + shared logic	Pure capability sharing

Unit 5

◆ What is an Error in Java?

An **error** is a **problem** that occurs during the execution of a program, which **stops it from working correctly**.

Java Errors are Mainly of 3 Types:

Type	Meaning	When it Happens
1. Syntax Error	Error in code writing rules (grammar mistake)	At compile-time
2. Runtime Error	Error while program is running	During execution
3. Logical Error	Program runs, but output is wrong	After execution

1. Syntax Error (Compile-Time Error)

- ◆ Caused by **wrong Java syntax**
- ◆ Detected by the compiler

Example:

```
java
Copy code
int a = 10
System.out.println(a); // ❌ Missing semicolon gives compile-time error
```

2. Runtime Error

- ◆ Code is syntactically correct
- ◆ But error occurs **during execution**

Common examples:

- `ArithmeticException` (divide by zero)
- `NullPointerException`
- `ArrayIndexOutOfBoundsException`

Example:

```
java
Copy code
int a = 10;
int b = 0;
System.out.println(a / b); // ❌ Runtime error: divide by zero
```

3. Logical Error

- ◆ Code runs without error
- ◆ But **wrong output** due to wrong logic

📌 Example:

```
java
Copy code
int a = 5, b = 10;
int avg = (a + b) / 2; // ✅ correct
// But if written: int avg = a + b / 2; ❌ wrong logic
```

📌 Output will be wrong, even though there's no error!

🧠 Summary Table:

Error Type	Detected When?	Example	Stops Program?
Syntax Error	Compile time	Missing ;	Yes
Runtime Error	While running	Divide by zero	Yes
Logical Error	After running	Wrong formula used	No (but wrong result)

◆ What is Abstraction?

Abstraction means **hiding the internal details** and showing **only the essential features** of an object.

It helps in **reducing complexity** and lets you **focus on what an object does** instead of *how* it does it.

🧠 Simple Example (Real-Life):

- When you drive a car, you just use the **steering, brakes, and accelerator**.
- You don't need to know how the engine or brakes **internally** work.

That's **abstraction** — you see only what's necessary.

Abstraction in Java

Java achieves abstraction in **two ways**:

1. **Abstract Class**
2. **Interface**

◆ 1. Using Abstract Class:

```
java
Copy code
abstract class Animal {
    abstract void sound(); // abstract method

    void sleep() {
        System.out.println("Sleeping...");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
```

◆ 2. Using Interface:

```
java
Copy code
interface Vehicle {
    void drive(); // abstract method
}

class Car implements Vehicle {
    public void drive() {
        System.out.println("Car is driving...");
    }
}
```

Key Points:

Feature	Explanation
Hides internal logic	Shows only necessary methods/behaviors
Achieved using	abstract class and interface

Feature	Explanation
Helps in	Reducing complexity & improving code security

What is Exception Handling?

Exception handling is used to **handle runtime errors** so the program doesn't crash suddenly.

Java provides **try**, **catch**, **finally** blocks to deal with exceptions.

1. try block

- Code that might throw an exception goes inside the `try` block.
- Java monitors this block for errors.

```
java
Copy code
try {
    // risky code here
}
```

2. catch block

- Used to handle the exception thrown by the `try` block.
- You can have **multiple catch blocks** for different exception types.

```
java
Copy code
catch(ExceptionType e) {
    // handle the exception
}
```

3. finally block

- This block **always executes**, whether an exception occurs or not.
- Used to **clean up resources**, like closing files or database connections.

```
java
Copy code
finally {
```

```
// cleanup code
}
```

Example Program:

```
java
Copy code
public class TryCatchExample {
    public static void main(String[] args) {
        try {
            int a = 10, b = 0;
            int result = a / b; // ⚠️ Will cause ArithmeticException
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("❌ Error: Cannot divide by zero!");
        } finally {
            System.out.println("✅ This is the finally block. Always runs.");
        }

        System.out.println("Program continues...");
    }
}
```

Output:

```
vbnet
Copy code
❌ Error: Cannot divide by zero!
✅ This is the finally block. Always runs.
Program continues...
```

Summary Table:

Block	Purpose	Runs When?
try	Code that might throw exception	Always
catch	Handles exception	Only if exception occurs
finally	Cleanup code (like closing files)	Always (even if exception occurs or not)

what is the used through key word explain with example

1. `throw` Keyword

► Used to manually throw an exception in Java.

✅ Syntax:

```
java
Copy code
throw new ExceptionType("Custom error message");
```

✅ Example:

```
java
Copy code
public class ThrowExample {
    public static void main(String[] args) {
        int age = 15;
        if (age < 18) {
            throw new ArithmeticException("❌ You are not eligible to vote");
        } else {
            System.out.println("✅ You can vote");
        }
    }
}
```

◆ 2. throws Keyword

► Used in method declaration to declare an exception that the method might throw.

✅ Syntax:

```
java
Copy code
void methodName() throws ExceptionType {
    // risky code
}
```

✅ Example:

```
java
Copy code
```

```
import java.io.*;

public class ThrowsExample {
    static void readFile() throws IOException {
        FileReader file = new FileReader("data.txt"); // risky code
        BufferedReader br = new BufferedReader(file);
        System.out.println(br.readLine());
    }

    public static void main(String[] args) {
        try {
            readFile();
        } catch (IOException e) {
            System.out.println("❌ File not found or error reading file");
        }
    }
}
```

Summary Table:

Keyword	Use	Where Used
throw	To manually throw an exception	Inside a method
throws	To declare possible exceptions a method can throw	In method signature

What is a Thread?

A **Thread** is a **lightweight sub-process**.
It's the **smallest unit of execution** in a program.

 **Thread = part of a program that runs independently**

Why Use Threads?

- For **multitasking** (e.g., downloading files while playing music)
 - To make programs **faster** and more **efficient**
-

Ways to Create a Thread in Java

Java provides **two main ways** to create a thread:

◆ 1. By Extending `Thread` class

```
java
Copy code
class MyThread extends Thread {
    public void run() {
        System.out.println("👑 King's thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start(); // starts the thread
    }
}
```

◆ 2. By Implementing `Runnable` Interface

```
java
Copy code
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("👑 Runnable thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        MyRunnable r = new MyRunnable();
        Thread t = new Thread(r); // wrap Runnable in Thread
        t.start(); // starts the thread
    }
}
```

🧠 Summary Table:

Method	Description
<code>extends Thread</code>	Subclass the <code>Thread</code> class
<code>implements Runnable</code>	Implement <code>Runnable</code> interface
<code>start()</code>	Starts the thread and calls <code>run()</code>
<code>run()</code>	Method that holds the thread's code

how do u create thread .explain both by extending thread by implementing run therad

1 By Extending the `Thread` Class

2 By Implementing the `Runnable` Interface

1 Creating Thread by Extending `Thread` Class

- You create a new class that **extends** `Thread`.
 - Override the `run()` method with the code you want to execute in the new thread.
 - Call the `start()` method to start the thread (which internally calls `run()`).
-

Example:

```
java

class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running by extending Thread class");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        t1.start(); // starts the thread, calls run()
    }
}
```

2 Creating Thread by Implementing `Runnable` Interface

- Create a class that **implements** `Runnable`.
 - Implement the `run()` method.
 - Create a `Thread` object passing your `Runnable` object.
 - Call `start()` on the `Thread` object.
-

Example:

```
java

class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread running by implementing Runnable
interface");
    }
}

public class Main {
    public static void main(String[] args) {
        MyRunnable r = new MyRunnable();
        Thread t1 = new Thread(r);
        t1.start(); // starts the thread, calls run()
    }
}
```

Key Differences:

Feature	Extending Thread	Implementing Runnable
Can extend other classes?	No (Java supports single inheritance)	Yes (can extend other classes)
Suitable for	Simple threading	More flexible & preferred
How to create thread object	Directly create object of subclass	Create Thread with Runnable

What is Thread Priority?

Thread Priority is a number assigned to a thread that **influences the order in which threads are scheduled to run** by the CPU.

- Threads with **higher priority** are more likely to be executed before threads with lower priority.
 - However, thread scheduling depends on the JVM and OS, so priority is a **hint**, not a strict rule.
-

Priority Levels in Java

- Priority is an **integer** between **1** and **10**.
- Java provides **three constants** in the `Thread` class for convenience:

Constant	Value	Meaning
<code>Thread.MIN_PRIORITY</code>	1	Lowest priority
<code>Thread.NORM_PRIORITY</code>	5	Default priority
<code>Thread.MAX_PRIORITY</code>	10	Highest priority

How to Set and Get Thread Priority?

- Use `setPriority(int priority)` to set a thread's priority.
 - Use `getPriority()` to get a thread's priority.
-

Example:

```
java

class MyThread extends Thread {
    public void run() {
        System.out.println(getName() + " with priority " + getPriority() + "
is running.");
    }
}

public class ThreadPriorityExample {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        MyThread t3 = new MyThread();

        t1.setName("Thread-1");
        t2.setName("Thread-2");
        t3.setName("Thread-3");

        t1.setPriority(Thread.MIN_PRIORITY);        // priority 1
        t2.setPriority(Thread.NORM_PRIORITY);        // priority 5
        t3.setPriority(Thread.MAX_PRIORITY);        // priority 10

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Possible Output:

```
Thread-1 with priority 1 is running.  
Thread-2 with priority 5 is running.  
Thread-3 with priority 10 is running.
```

Remember, the order of output may vary because thread scheduling depends on the JVM and OS.

Summary:

Concept	Description
Thread Priority	Indicates thread importance
Range	1 (lowest) to 10 (highest)
Default Priority	5 (<code>NORM_PRIORITY</code>)
Effect	Helps scheduler decide order, but not guaranteed

What is Inter-thread Communication?

When multiple threads share resources, sometimes threads need to **communicate and coordinate** with each other — for example, one thread waits until another finishes a task or produces data.

Inter-thread communication allows threads to **communicate about the state of shared resources** to avoid problems like race conditions or deadlocks.

Why is Inter-thread Communication Important?

- Prevents **race conditions** where threads try to access shared data simultaneously.
 - Helps threads **wait and notify** each other when a resource is available or a task is done.
 - Ensures **proper sequencing** of operations among threads.
 - Improves **program efficiency** by avoiding unnecessary polling or busy-waiting.
-

Key Methods for Inter-thread Communication

These methods are part of the **Object class** (because every object in Java can be used as a communication lock):

Method	Purpose
<code>wait()</code>	Makes the current thread release the lock and wait until notified
<code>notify()</code>	Wakes up one waiting thread on the same object's lock
<code>notifyAll()</code>	Wakes up all waiting threads on the same object's lock

How `wait()` and `notify()` Work Together

- A thread calls `wait()` on an object to pause itself and release the object's lock.
 - Another thread calls `notify()` or `notifyAll()` on the same object to wake waiting threads.
 - After waking up, the waiting thread reacquires the lock and continues.
-

Example of `wait()` and `notify()`:

```
java

class Data {
    private int number;
    private boolean available = false;

    public synchronized void produce(int num) {
        while (available) {
            try {
                wait(); // wait if data is already available
            } catch (InterruptedException e) {}
        }
        number = num;
        available = true;
        System.out.println("Produced: " + number);
        notify(); // notify consumer
    }

    public synchronized void consume() {
        while (!available) {
            try {
                wait(); // wait if no data available
            } catch (InterruptedException e) {}
        }
        System.out.println("Consumed: " + number);
        available = false;
        notify(); // notify producer
    }
}
```



```

class Producer implements Runnable {
    Data data;
    Producer(Data d) { data = d; }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            data.produce(i);
        }
    }
}

class Consumer implements Runnable {
    Data data;
    Consumer(Data d) { data = d; }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            data.consume();
        }
    }
}

public class InterThreadCommunicationExample {
    public static void main(String[] args) {
        Data data = new Data();
        Thread producer = new Thread(new Producer(data));
        Thread consumer = new Thread(new Consumer(data));

        producer.start();
        consumer.start();
    }
}

```

What is Deadlock?

A **deadlock** occurs when **two or more threads are blocked forever**, each waiting for a resource held by the other, so **none of them can proceed**.

It's like a standstill where threads are stuck, causing your program to freeze.

Example (Simple Explanation):

- Thread A holds **Resource 1** and waits for **Resource 2**.
- Thread B holds **Resource 2** and waits for **Resource 1**.
- Neither can proceed because each is waiting for the other to release the resource.

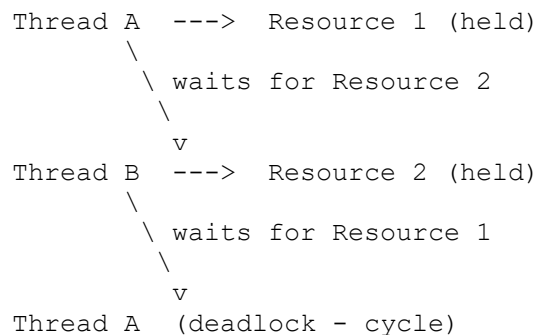
◆ Conditions for Deadlock (The 4 Coffman Conditions)

All **four conditions** must be true for a deadlock to occur:

Condition Name	Explanation
1. Mutual Exclusion	Resources cannot be shared; only one thread at a time can use a resource.
2. Hold and Wait	A thread holding at least one resource is waiting to acquire additional resources held by others.
3. No Preemption	Resources cannot be forcibly taken from a thread; they must be released voluntarily.
4. Circular Wait	A closed chain of threads exists, where each thread waits for a resource held by the next thread in the chain.

Visual Summary:

rust



How to Avoid Deadlock?

- Avoid **circular wait** by ordering resource requests.
- Use **timeout** when waiting for resources.
- Use **deadlock detection algorithms**.
- Acquire all resources at once if possible.

◆ What is Synchronization?

Synchronization in Java is a mechanism that **controls access to shared resources by multiple threads**. It ensures that **only one thread can access a resource at a time**, preventing data inconsistency and race conditions.

Why Synchronization is Important?

- When multiple threads access and modify shared data simultaneously, it can cause **unexpected or incorrect results**.
 - Synchronization **prevents thread interference** and **maintains data consistency**.
-

Advantages of Synchronization

Advantage	Explanation
Prevents Data Inconsistency	Ensures shared data is accessed by one thread at a time
Avoids Race Conditions	Prevents threads from changing data unpredictably
Guarantees Thread Safety	Makes code safe to run in a multithreaded environment
Helps in Coordination	Coordinates threads that depend on shared data

How to Use Synchronization in Java?

You use the **synchronized** keyword to mark a method or a block of code that **only one thread can execute at a time**.

Synchronizing a Method:

```
java
Copy code
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
        System.out.println("Count: " + count);
    }
}
```

Synchronizing a Block:

```
java
Copy code
class Counter {
    private int count = 0;

    public void increment() {
        synchronized(this) {
            count++;
            System.out.println("Count: " + count);
        }
    }
}
```

Example Program:

```
java
Copy code
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
        System.out.println(Thread.currentThread().getName() + " : " + count);
    }
}

public class SyncExample {
    public static void main(String[] args) {
        Counter counter = new Counter();

        Runnable task = () -> {
            for (int i = 0; i < 5; i++) {
                counter.increment();
                try {
                    Thread.sleep(50); // simulate some work
                } catch (InterruptedException e) {}
            }
        };

        Thread t1 = new Thread(task, "Thread-1");
        Thread t2 = new Thread(task, "Thread-2");

        t1.start();
        t2.start();
    }
}
```

Explanation:

- Both threads share the same Counter object.

- The `increment()` method is synchronized, so only one thread can increase `count` at a time.
 - This prevents **race conditions** and ensures accurate output.
-

write a program to add to matrix add,multiply ,subtract transpose

```
import java.util.Scanner;
```

```
public class MatrixOperations {
```

```
    // Method to add two matrices
```

```
    public static int[][] add(int[][] a, int[][] b) {
```

```
        int rows = a.length;
```

```
        int cols = a[0].length;
```

```
        int[][] result = new int[rows][cols];
```

```
        for(int i=0; i<rows; i++){
```

```
            for(int j=0; j<cols; j++){
```

```
                result[i][j] = a[i][j] + b[i][j];
```

```
            }
```

```
        }
```

```
        return result;
```

```
    }
```

```
    // Method to subtract two matrices
```

```
    public static int[][] subtract(int[][] a, int[][] b) {
```

```
        int rows = a.length;
```

```
        int cols = a[0].length;
```

```
        int[][] result = new int[rows][cols];
```

```
        for(int i=0; i<rows; i++){
```

```
            for(int j=0; j<cols; j++){
```

```
                result[i][j] = a[i][j] - b[i][j];
```

```
            }
```

```
        }
```

```
        return result;
```

```
    }
```

```
    // Method to multiply two matrices
```

```
    public static int[][] multiply(int[][] a, int[][] b) {
```

```
        int rowsA = a.length;
```

```
        int colsA = a[0].length;
```

```

int rowsB = b.length;
int colsB = b[0].length;

if(colsA != rowsB){
    throw new IllegalArgumentException("Cannot multiply matrices: incompatible
dimensions.");
}

int[][] result = new int[rowsA][colsB];

for(int i=0; i<rowsA; i++){
    for(int j=0; j<colsB; j++){
        result[i][j] = 0;
        for(int k=0; k<colsA; k++){
            result[i][j] += a[i][k] * b[k][j];
        }
    }
}
return result;
}

```

```

// Method to get transpose of a matrix
public static int[][] transpose(int[][] a) {
    int rows = a.length;
    int cols = a[0].length;
    int[][] result = new int[cols][rows];

    for(int i=0; i<rows; i++){
        for(int j=0; j<cols; j++){
            result[j][i] = a[i][j];
        }
    }
    return result;
}

```

```

// Utility method to print matrix
public static void printMatrix(int[][] matrix) {
    for(int[] row : matrix){
        for(int val : row){
            System.out.print(val + "\t");
        }
        System.out.println();
    }
}

```

```

public static void main(String[] args){
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of rows for matrix A: ");
    int rowsA = sc.nextInt();
    System.out.print("Enter number of columns for matrix A: ");
    int colsA = sc.nextInt();

    int[][] matrixA = new int[rowsA][colsA];

    System.out.println("Enter elements for matrix A:");
    for(int i=0; i<rowsA; i++){
        for(int j=0; j<colsA; j++){
            matrixA[i][j] = sc.nextInt();
        }
    }

    System.out.print("Enter number of rows for matrix B: ");
    int rowsB = sc.nextInt();
    System.out.print("Enter number of columns for matrix B: ");
    int colsB = sc.nextInt();

    int[][] matrixB = new int[rowsB][colsB];

    System.out.println("Enter elements for matrix B:");
    for(int i=0; i<rowsB; i++){
        for(int j=0; j<colsB; j++){
            matrixB[i][j] = sc.nextInt();
        }
    }

    // Addition and Subtraction possible only if dimensions are same
    if(rowsA == rowsB && colsA == colsB) {
        System.out.println("\nMatrix A + Matrix B:");
        printMatrix(add(matrixA, matrixB));

        System.out.println("\nMatrix A - Matrix B:");
        printMatrix(subtract(matrixA, matrixB));
    } else {
        System.out.println("\nAddition and Subtraction not possible due to dimension
mismatch.");
    }
}

```

```

// Multiplication possible only if cols of A = rows of B
if(colsA == rowsB) {
    System.out.println("\nMatrix A * Matrix B:");
    printMatrix(multiply(matrixA, matrixB));
} else {
    System.out.println("\nMultiplication not possible due to dimension mismatch.");
}

// Transpose of Matrix A
System.out.println("\nTranspose of Matrix A:");
printMatrix(transpose(matrixA));

// Transpose of Matrix B
System.out.println("\nTranspose of Matrix B:");
printMatrix(transpose(matrixB));

sc.close();
}
}

```

what is difference between array list & array?

Feature	Array	ArrayList
Size	Fixed size (must define at creation)	Dynamic size (can grow/shrink)
Type	Can hold primitives and objects	Can hold only objects (no primitives)
Performance	Faster (less overhead)	Slightly slower (due to dynamic resizing and boxing/unboxing)
Methods	Limited (basic access and length)	Rich set of methods (add, remove, search, etc.)
Syntax	<code>int[] arr = new int[5];</code>	<code>ArrayList<Integer> list = new ArrayList<>();</code>
Usage	Use when size is fixed and known	Use when size changes dynamically
Type Safety	Supports primitive types (int, char) directly	Works with objects only (use wrapper classes for primitives)

Quick Example:


```
java

// Array
int[] arr = new int[3];
arr[0] = 10;

// ArrayList
ArrayList<Integer> list = new ArrayList<>();
list.add(10);
list.add(20);
```

◆ What is AWT?

AWT stands for **Abstract Window Toolkit**.

- It's Java's **original GUI (Graphical User Interface) toolkit**.
 - Provides basic components like buttons, text fields, labels, and windows.
 - Uses **native system GUI components** (called "**heavyweight**" components).
 - Because it relies on the operating system's native components, the look and feel depends on the platform (Windows, Mac, Linux).
 - Limited in functionality and customization.
-

◆ What is Swing?

Swing is a **GUI toolkit built on top of AWT**.

- Provides a **richer set of GUI components** like tables, trees, sliders, tabbed panes, and more.
- Uses **lightweight components** — completely written in Java, so they look the same across platforms (platform-independent).
- More flexible and customizable — supports pluggable look and feel (you can make your app look like Windows, Mac, or custom style).
- Swing components start with **J** (e.g., JButton, JFrame, JLabel).

Example:

AWT Button:

```
java

import java.awt.*;
import java.awt.event.*;

public class AWTEExample {
    public static void main(String[] args) {
        Frame frame = new Frame("AWT Example");
        Button button = new Button("Click Me");

        button.setBounds(50,100,80,30);
        frame.add(button);

        frame.setSize(200,200);
        frame.setLayout(null);
        frame.setVisible(true);

        frame.addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }
}
```

Swing Button:

```
java

import javax.swing.*;
import java.awt.event.*;

public class SwingExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Swing Example");
        JButton button = new JButton("Click Me");

        button.setBounds(50,100,80,30);
        frame.add(button);

        frame.setSize(200,200);
        frame.setLayout(null);
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

Q.write a program to create frame

```
import javax.swing.JFrame;
```

```
public class SimpleFrame {  
    public static void main(String[] args) {  
        // Create a new JFrame instance  
        JFrame frame = new JFrame("My First Frame");  
  
        // Set the size of the frame (width, height)  
        frame.setSize(400, 300);  
  
        // Specify what happens when the user closes the frame  
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
  
        // Make the frame visible  
        frame.setVisible(true);  
    }  
}
```